

JavaScript Function Return

[Back to JS page](#)

Table of Contents

- [JavaScript Function Return](#)
 - [The return Statement](#)
 - [Example 1](#)
 - [Returning a Calculated Value](#)
 - [Example 2](#)
 - [Using Return Values in Expressions](#)
 - [Example 3](#)
 - [Return Statements Stop Execution](#)
 - [Example 4](#)
 - [Functions Without return](#)
 - [Example 5](#)
 - [Returning Values Early](#)
 - [Example 6](#)
 - [Document](#)
 - [Reference](#)

The return Statement

When JavaScript reaches a `return` statement, the function will stop executing.

If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The return Statement</h4>
<p id="demo"></p>

<script>
function myFunction(a, b) {
  return a * b;
}
document.getElementById("demo").innerHTML = myFunction(4, 3);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Returning a Calculated Value

A function can return a calculated value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Returning a Calculated Value</h4>
<p id="demo"></p>

<script>
function calculateArea(width, height) {
    return width * height;
}
let area = calculateArea(5, 10);
document.getElementById("demo").innerHTML = "Area: " + area;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Using Return Values in Expressions

The return value of a function can be used directly in expressions:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using Return Values in Expressions</h4>
<p id="demo"></p>

<script>
function add(x, y) {
    return x + y;
}
// Using return value directly in expression
let result = add(5, 10) * 2;
document.getElementById("demo").innerHTML = "(5 + 10) * 2 = " + result;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Return Statements Stop Execution

After the `return` statement is executed, the rest of the code in the function does NOT execute:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Return Statements Stop Execution</h4>
<p id="demo"></p>

<script>
function myFunction() {
  return "This is returned";
  // Code after return will NOT execute
  let x = 5; // This line is ignored
}
document.getElementById("demo").innerHTML = myFunction();
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Functions Without return

A function without a `return` statement will return `undefined` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions Without return</h4>
<p id="demo"></p>

<script>
function myFunction(a, b) {
  a + b;
  // No return statement
}
let result = myFunction(5, 3);
document.getElementById("demo").innerHTML = "Return value: " + result;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Returning Values Early

You can use `return` to exit a function early when a condition is met:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Return</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Returning Values Early</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function checkAge(age) {
  if (age < 18) {
    return "Too young!";
  }
  return "Access granted";
}
document.getElementById("demo1").innerHTML = "Age 15: " + checkAge(15);
document.getElementById("demo2").innerHTML = "Age 21: " + checkAge(21);
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Function Return](#)